UrbanElectra

Full-Stack E-Commerce Platform

Complete Architecture & Workflow Documentation

MERN Stack • Stripe • Redis • Cloudinary • Vercel

https://urban-electra.vercel.app

1. Project Overview

UrbanElectra is a production-ready, full-stack electronics e-commerce platform. It provides a complete shopping experience for customers and a rich admin panel for store management.

Tech Stack

Frontend	React 19 + Vite, Tailwind CSS v4, Zustand, Framer Motion, Recharts
Backend	Node.js, Express 5, MongoDB Atlas (Mongoose)
Auth	JWT Access Tokens (15 min) + Refresh Tokens (7 days) stored in HttpOnly cookies
Cache	Upstash Redis via ioredis – stores refresh tokens & featured-product cache
Payments	Stripe Checkout Sessions + Stripe Refunds API
Storage	Cloudinary – product image upload & delivery
Deployment	Vercel (frontend dist + serverless API via /api/index.js)

Folder Structure

```
urbanelectra/
├── api/
│   └── index.js              ← Vercel serverless entry (re-exports backend app)
├── backend/
│   ├── controllers/          ← Route logic (auth, cart, product, order, payment, analytics)
│   ├── lib/                  ← DB, Redis, Cloudinary, Stripe initialisation
│   ├── middleware/           ← protectRoute, adminRoute
│   ├── models/               ← Mongoose schemas (User, Product, Order, Coupon)
│   ├── routes/               ← Express routers
│   └── server.js             ← App factory + middleware setup
├── frontend/
│   └── src/
│       ├── components/       ← Reusable UI pieces
│       ├── pages/            ← Route-level pages
│       ├── stores/           ← Zustand global state
│       └── lib/axios.js      ← Axios instance with interceptors
├── package.json              ← Root (backend deps + build script)
└── vercel.json               ← Build config + API/SPA rewrites
```

2. Request Lifecycle (End-to-End)

Every HTTP call travels this path:

1	**Browser / React Component** Calls an axios method from a Zustand store function.
2	**Axios Instance (lib/axios.js)** Prepends the base URL (/api in prod, http://localhost:5000/api in dev). Attaches cookies automatically (withCredentials: true).
3	**Vercel Rewrite / Express Router** All /api/* requests hit api/index.js → backend/server.js. Express routes the request to the correct controller.
4	**Auth Middleware (protectRoute)** Reads accessToken cookie, verifies JWT. If expired, the frontend interceptor auto-retries after calling /api/auth/refresh-token.
5	**Controller Function** Executes business logic, queries MongoDB / Redis / Stripe / Cloudinary.
6	**JSON Response** Returns data to Zustand store, which updates React state and triggers a re-render.

Token Refresh Interceptor

The Axios response interceptor (useUserStore.js) silently handles 401 errors:

```
On 401:
  1. Mark originalRequest._retry = true
  2. Call POST /api/auth/refresh-token (sends refreshToken cookie)
  3. Server validates token against Redis, issues new accessToken cookie
  4. Retry the original request automatically
  5. If refresh also fails → logout user
```

3. Authentication & Authorization

Sign-Up Flow

1	**User fills SignUpPage.jsx** name, email, password, confirmPassword validated client-side in useUserStore.
2	**POST /api/auth/signup** Server checks for existing email, creates User with bcrypt-hashed password.
3	**Tokens generated** generateTokens() creates accessToken (15 min JWT) + refreshToken (7 day JWT).
4	**Redis storage** storeRefreshToken() stores refresh_token:{userId} → token in Upstash with 7-day TTL.
5	**HttpOnly cookies set** setCookies() writes accessToken + refreshToken as secure, sameSite=strict cookies.
6	**User object returned** { _id, name, email, role } stored in Zustand useUserStore → user.

Login Flow

Same as signup except: User.findOne({ email }) → comparePassword (bcrypt.compare) → generate & store tokens → set cookies.

Logout Flow

Decodes refreshToken cookie → deletes Redis key → clears both cookies → Zustand sets user: null.

Route Protection

protectRoute middleware	Reads accessToken cookie, jwt.verify() → attaches User document to req.user
adminRoute middleware	Checks req.user.role === 'admin'. Returns 403 if not admin.
Frontend guard	App.jsx uses <Navigate to='/login'> for protected routes when user is null; <Navigate to='/'> for admin route when user.role ≠ 'admin'

4. Frontend Pages & Routing (App.jsx)

/ (HomePage)	Public – hero categories grid + FeaturedProducts carousel
/signup (SignUpPage)	Public – redirects to / if already logged in
/login (LoginPage)	Public – redirects to / if already logged in
/category/:category	Public – CategoryPage showing filtered products
/cart (CartPage)	Protected – cart items, order summary, coupon, address form
/purchase-success	Protected – PurchaseSuccessPage (Stripe redirect after pay)
/purchase-cancel	Protected – PurchaseCancelPage
/my-orders (OrdersPage)	Protected (non-admin) – order history & management
/secret-dashboard (AdminPage)	Admin only – tabbed dashboard

Page-by-Page Breakdown

HomePage.jsx

- On mount: fetchFeaturedProducts() → GET /api/products/featured → checks Redis cache first
- Renders 7 CategoryItem cards (phones, laptops, tablets, headphones, cameras, tvs, accessories)
- Renders FeaturedProducts carousel if products.length > 0

CategoryPage.jsx

- Reads :category param → fetchProductsByCategory() → GET /api/products/category/:category
- Maps products array to ProductCard components

CartPage.jsx

- Left column: CartItem list + PeopleAlsoBought (random 4 recommendations)
- Right column: OrderSummary → GiftCouponCard → AddressForm
- Checkout button disabled until shippingAddress is confirmed

OrdersPage.jsx

- Fetches GET /api/orders/my-orders on mount
- Each order renders as a collapsible OrderCard
- OrderCard shows: items, tracking, shipping address, status timeline
- Action buttons: Cancel (pending/confirmed), Return/Exchange (delivered)
- ActionModal captures optional reason text before confirming

AdminPage.jsx

- 4 tabs: Create Product | Products | Orders | Analytics
- Create Product → CreateProductForm (base64 image → Cloudinary upload)
- Products → ProductsList with star-toggle (featured) and delete
- Orders → OrdersTab (full order management)
- Analytics → AnalyticsTab (stat cards + daily line chart)

5. Components Deep-Dive

Navbar.jsx

Fixed top bar. Shows: UrbanElectra logo, Home, Cart (with item count badge), My Orders (non-admin), Dashboard (admin), Logout/Login/SignUp.

FeaturedProducts.jsx

Sliding carousel. Calculates itemsPerPage by window width (1/2/3/4). Prev/Next buttons disabled at boundaries. Each card has 'Add to Cart' button calling useCartStore.addToCart().

ProductCard.jsx

Used in CategoryPage and PeopleAlsoBought. Requires login to add to cart (toast warning if not logged in).

CartItem.jsx

Shows product image, name, description, price. +/- buttons call updateQuantity(). Trash icon calls removeFromCart(). When quantity hits 0, item is removed.

AddressForm.jsx

Multi-field shipping address form with validation. US users get a state dropdown; other countries get a text input. On confirm, calls useCartStore.setShippingAddress(). Shows a summary card once confirmed with an Edit link.

OrderSummary.jsx

Reads cart, coupon, shippingAddress from useCartStore. Displays subtotal, savings, coupon discount, and total. Checkout button calls POST /api/payments/create-checkout-session with products + couponCode + shippingAddress, then redirects to Stripe.

GiftCouponCard.jsx

Fetches user's active coupon (GET /api/coupons) on mount. Validates and applies via POST /api/coupons/validate. Triggers calculateTotals() after apply/remove.

AnalyticsTab.jsx

Fetches GET /api/analytics on mount. Shows 6 stat cards: Total Users, Total Products, Completed Sales, Net Revenue, Refunded Orders, Refunded Amount. Recharts LineChart shows last 7 days of sales and revenue (dual Y-axis).

OrdersTab.jsx (Admin)

Full order management UI. Includes:

- Search by customer name/email
- Status filter buttons with live counts
- Table with inline status dropdown, tracking modal, refund modal, detail drawer
- Expandable rows with quick info (shipping, tracking, reasons, refund)
- Pagination for large order sets

CreateProductForm.jsx

FileReader converts uploaded image to base64. POSTs to /api/products. Server uploads base64 to Cloudinary and stores the secure_url.

6. Zustand State Stores

useUserStore.js

State	user, loading, checkingAuth
signup()	POST /auth/signup → set user
login()	POST /auth/login → set user
logout()	POST /auth/logout → set user: null
checkAuth()	GET /auth/profile on app load → restores session from cookie
refreshToken()	POST /auth/refresh-token → used by axios interceptor silently

useCartStore.js

State	cart[], coupon, total, subtotal, isCouponApplied, shippingAddress
getCartItems()	GET /cart → populate cart array
addToCart()	POST /cart → optimistic update in state
removeFromCart()	DELETE /cart with productId body
updateQuantity()	PUT /cart/:id with quantity
applyCoupon()	POST /coupons/validate → set coupon + recalculate
calculateTotals()	Pure: subtotal = Σ(price×qty); applies coupon % discount
clearCart()	Resets cart, coupon, totals, shippingAddress after successful order

useProductStore.js

State	products[], loading
fetchFeaturedProducts()	GET /products/featured
fetchProductsByCategory()	GET /products/category/:cat
fetchAllProducts()	GET /products (admin only)
createProduct()	POST /products (admin)
deleteProduct()	DELETE /products/:id (admin)
toggleFeaturedProduct()	PATCH /products/:id (admin)

useOrderStore.js

State	orders[], adminOrders[], orderStats{}, loading, pagination
fetchMyOrders()	GET /orders/my-orders
cancelOrder()	POST /orders/:id/cancel
requestReturn()	POST /orders/:id/return
requestExchange()	POST /orders/:id/exchange
fetchAllOrders()	GET /orders/admin/all with filters/pagination
fetchOrderStats()	GET /orders/admin/stats
updateOrderStatus()	PATCH /orders/admin/:id/status
setTrackingInfo()	PATCH /orders/admin/:id/tracking
processRefund()	POST /orders/admin/:id/refund → calls Stripe Refunds API

7. Backend API Routes

Auth Routes /api/auth

POST /signup	Create user, generate tokens, set cookies
POST /login	Verify credentials, generate tokens, set cookies
POST /logout	Delete Redis refresh token, clear cookies
POST /refresh-token	Validate refresh token in Redis, issue new access token cookie
GET /profile	protectRoute → return req.user

Product Routes /api/products

GET /featured	Public – Redis cache first, then MongoDB, cache result
GET /category/:category	Public – find by category
GET /recommendations	Public – $sample 4 random products
GET /	Admin – all products
POST /	Admin – create, upload image to Cloudinary
PATCH /:id	Admin – toggle isFeatured, update Redis cache
DELETE /:id	Admin – delete product + Cloudinary image

Cart Routes /api/cart

GET /	Protected – return populated cart products
POST /	Protected – add product or increment quantity in user.cartItems
DELETE /	Protected – remove one product or clear all (body: { productId })
PUT /:id	Protected – set specific quantity

Coupon Routes /api/coupons

GET /	Protected – user's active coupon
POST /validate	Protected – validate + return discount percentage

Payment Routes /api/payments

POST /create-checkout-session	Build Stripe line items, apply coupon discount, store metadata (products, userId, shippingAddress), create Stripe session → return URL
POST /checkout-success	Retrieve Stripe session, verify paid, deactivate coupon, create Order document with status 'pending'

Order Routes /api/orders

GET /my-orders	Protected – user's own orders
GET /:id	Protected – single order (user owns it or admin)
POST /:id/cancel	Protected – cancel if pending/confirmed
POST /:id/return	Protected – request return if delivered + within 30 days
POST /:id/exchange	Protected – request exchange if delivered
GET /admin/all	Admin – paginated, filterable order list
GET /admin/stats	Admin – count/revenue grouped by status
PATCH /admin/:id/status	Admin – update status, append to statusHistory
PATCH /admin/:id/tracking	Admin – set tracking info, auto-set status to 'shipped'
POST /admin/:id/refund	Admin – Stripe refund + update order

Analytics Routes /api/analytics

GET /	Admin – aggregate user/product counts, sales/revenue (excl. cancelled/refunded/returned), refund totals, + daily sales for last 7 days

8. Database Models

User

Fields	name, email (unique, lowercase), password (bcrypt), cartItems [{product ref, quantity}], role (customer|admin)
Hooks	pre-save: hash password if modified
Methods	comparePassword(plain) → bcrypt.compare

Product

Fields	name, description, price, image (Cloudinary URL), category, isFeatured

Order

Core	user ref, products [{product ref, quantity, price}], totalAmount, stripeSessionId (unique)
Shipping	shippingAddress sub-document: fullName, addressLine1/2, city, state, postalCode, country, phone
Status	enum: pending→confirmed→processing→shipped→delivered / cancelled / return_requested→returned / exchange_requested→exchanged / refunded
History	statusHistory [{status, changedAt, note}]
Tracking	trackingNumber, trackingCarrier, estimatedDelivery
Reasons	cancelReason, returnReason, exchangeReason
Refund	stripeRefundId, refundAmount, refundedAt

Coupon

Fields	code (unique), discountPercentage, expirationDate, isActive, userId ref (unique – one coupon per user)
Logic	Created automatically when order subtotal ≥ $200 (GIFT + random 6-char code, 10% off, 30-day expiry). Deactivated on use.

9. Stripe Payment Flow

1	**User clicks 'Proceed to Checkout'** OrderSummary.jsx POSTs to /api/payments/create-checkout-session with cart products, optional coupon code, and confirmed shipping address.
2	**Server builds Stripe Session** Creates line_items from products, applies Stripe coupon if discount code valid, embeds userId + products JSON + shippingAddress JSON in session metadata (max 500 chars). Returns session URL.
3	**Browser redirects to Stripe Checkout** User enters card details on Stripe-hosted page.
4	**Stripe redirects to /purchase-success?session_id=xxx** PurchaseSuccessPage.jsx calls POST /api/payments/checkout-success with sessionId.
5	**Server verifies & creates Order** Retrieves session from Stripe API, checks payment_status === 'paid', deactivates coupon, parses products from metadata, creates Order document with status 'pending'. Guards against duplicates via stripeSessionId unique index.
6	**Cart cleared + confetti** Frontend clears cart in Zustand, shows confetti animation, displays order ID with link to My Orders.
7	**Admin refund** Admin clicks refund in OrdersTab → POST /api/orders/admin/:id/refund → retrieves payment_intent from Stripe session → creates Stripe refund → stores stripeRefundId, refundAmount, updates status to 'refunded'.

Loyalty Coupon Trigger

If the original cart subtotal (before coupon) is ≥ $200, createNewCoupon() is called in createCheckoutSession: deletes any existing coupon for the user, creates a new GIFT+XXXXXX code (10% off, 30 days). This runs before the session is returned so it is always created, even if checkout is abandoned.

10. Order Lifecycle & Status Flow

Every status change appends an entry to statusHistory[] with timestamp and note.

pending	Order created after successful payment
confirmed	Admin acknowledges the order
processing	Order is being picked/packed
shipped	Admin sets tracking info (auto-transitions from confirmed/processing)
delivered	Admin marks as delivered
cancelled	Customer cancels (from pending/confirmed only) OR admin cancels
return_requested	Customer requests return within 30 days of delivery
returned	Admin marks return accepted
exchange_requested	Customer requests exchange (from delivered)
exchanged	Admin marks exchange completed
refunded	Admin processes Stripe refund

Customer Permissions

- Cancel: only when status is 'pending' or 'confirmed'
- Return: only when status is 'delivered' AND within 30 days of delivery timestamp in statusHistory
- Exchange: only when status is 'delivered'

Admin Permissions

- Can set any valid status via dropdown in OrdersTab
- Can add/edit tracking info (auto-promotes to 'shipped')
- Can process partial or full Stripe refunds

11. Redis (Upstash) Usage

refresh_token:{userId}	Stores the refresh token string with 7-day TTL. Validated on every /auth/refresh-token call to prevent token reuse after logout.
featured_products	JSON-stringified array of featured products. Read first on GET /products/featured. Updated (overwritten) every time toggleFeaturedProduct() is called in the admin panel.

Connection uses ioredis with TLS (rejectUnauthorized: false for Upstash rediss:// URLs) and maxRetriesPerRequest: 3. Redis errors are caught silently in storeRefreshToken() to prevent auth failures from cascading.

12. Deployment (Vercel)

Build Process

```
npm run build:
  1. npm install (root backend deps)
  2. npm install --prefix frontend --include=dev
  3. npm run build --prefix frontend  →  frontend/dist/
```

vercel.json Rewrites

/api/(.*)	Routed to /api/index.js (serverless function wrapping Express app)
/(.*)	All other paths served from frontend/dist/index.html (SPA fallback)

Environment Variables Required

MONGO_URI	MongoDB Atlas connection string
ACCESS_TOKEN_SECRET	JWT secret for access tokens
REFRESH_TOKEN_SECRET	JWT secret for refresh tokens
UPSTASH_REDIS_URL	rediss://... URL from Upstash
CLOUDINARY_CLOUD_NAME	Cloudinary cloud name
CLOUDINARY_API_KEY	Cloudinary API key
CLOUDINARY_API_SECRET	Cloudinary API secret
STRIPE_SECRET_KEY	Stripe secret key (sk_live_ or sk_test_)
CLIENT_URL	Frontend URL for Stripe success/cancel redirects and CORS
NODE_ENV	Set to 'production' on Vercel

13. User vs Admin Experience

Regular Customer Journey

1	**Land on HomePage** Browse featured products and category tiles.
2	**Browse a Category** Click CategoryItem → CategoryPage → ProductCards.
3	**Add to Cart** Click 'Add to Cart' (must be logged in) → item appears in cart badge.
4	**Go to Cart** Review CartItem list, adjust quantities, see PeopleAlsoBought suggestions.
5	**Apply Coupon** Enter coupon code in GiftCouponCard → discount applied to total.
6	**Enter Shipping Address** Fill AddressForm → click Confirm Address → OrderSummary unlocks checkout.
7	**Checkout** Click 'Proceed to Checkout' → Stripe-hosted payment page.
8	**Post-payment** Redirected to /purchase-success → confetti, order ID shown, cart cleared.
9	**Track Orders** Visit /my-orders → expand any OrderCard for full details, timeline, tracking.
10	**Cancel/Return/Exchange** Buttons appear contextually based on order status. ActionModal captures reason.

Admin Journey

1	**Login with admin account** role: 'admin' in MongoDB. Navbar shows Dashboard button instead of My Orders.
2	**Open /secret-dashboard** AdminPage with 4 tabs loads. fetchAllProducts() called on mount.
3	**Create Product tab** Fill form, upload image → base64 → Cloudinary → Product document created.
4	**Products tab** ProductsList shows all products. Star toggles featured status (updates Redis cache). Trash deletes product + Cloudinary image.
5	**Orders tab** OrdersTab loads all orders with stats. Filter by status or search by customer.
6	**Manage an Order** Change status via dropdown, set tracking info (opens modal), process refund (opens modal with optional partial amount).
7	**Analytics tab** AnalyticsTab shows aggregate stats cards and a 7-day sales/revenue line chart.

14. Security Highlights

HttpOnly Cookies	Tokens never accessible via JS. Prevents XSS token theft.
sameSite: strict	Cookies not sent on cross-site requests. Prevents CSRF.
secure: true (prod)	Cookies only sent over HTTPS in production.
Redis token validation	Server validates refresh token against stored value, preventing replay attacks after logout.
bcryptjs (salt 10)	Passwords stored as salted hashes. comparePassword used for login.
Role-based guards	adminRoute middleware on all admin endpoints. Frontend route guards match.
10 MB body limit	express.json({ limit: '10mb' }) supports base64 image uploads without being too permissive.
Idempotent checkout	stripeSessionId unique index prevents duplicate orders if success webhook fires twice.
Refund via Stripe API	Refunds go through official Stripe Refunds API using the payment_intent from the session.